

PROJECT REPORT

**"SMART RESUME ANALYZER WITH
AI-BASED FEEDBACK"**

Artificial Intelligence & Natural Language Processing Project

Submitted By
Subbaraju Karlapalem

Project Provided By
SkillOrbit

Project Year: 2026

DECLARATION

I, Subbaraju Karlapalem, hereby declare that the project titled "Smart Resume Analyzer with AI-Based Feedback" is my project work carried out as part of the project assigned by SkillOrbit. The project demonstrates the design, development, document parsing, deterministic ATS compatibility screening, weighted scoring with relevance guardrails, containerized deployment, and client-side vector PDF reporting using modern cloud-oriented technologies.

The work presented in this report is prepared for academic and project evaluation purposes and represents the implementation completed during the project.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to SkillOrbit for providing the opportunity to work on the project "Smart Resume Analyzer with AI-Based Feedback." This project provided practical exposure to natural language text extraction, regular expression boundary tokenization, multi-factor scoring algorithms, modular web application architecture, continuous deployment, and production WSGI containerization.

I also acknowledge the contribution of the tools and technologies used during the development of the project, including Python Flask, pypdf, python-docx, Gunicorn, GitHub, and Render. Working with these technologies helped in understanding how an industry-aligned recruitment screening tool can be engineered, tested, and deployed using modern cloud engineering practices.

ABSTRACT

The project "Smart Resume Analyzer with AI-Based Feedback" focuses on developing and deploying an automated, web-based recruitment analytics application using modern web and natural language processing technologies. The application is implemented using Python Flask as the backend framework and pypdf and python-docx for document parsing. It provides multi-format resume ingestion, contact and section integrity verification, target role keyword compatibility matching, and exportable vector PDF reporting.

To improve parsing fidelity and eliminate the false-positive substring matches common in traditional ATS checkers, the system incorporates a custom lookaround boundary regular expression engine that accurately isolates engineering tokens containing punctuation such as C++, Node.js, and CI/CD without boundary bleeding.

A multi-factor scoring model is implemented that combines role keyword match, technical depth, section architecture, and contact completeness. To prevent artificial score inflation where unqualified resumes receive passing scores merely for possessing valid contact headers, strict relevance guardrails are enforced (capping zero-match resumes at $\leq 25/100$).

A comprehensive feedback engine is also implemented to guide candidates using Google's X-Y-Z bullet point formula ("Accomplished [X] as measured by [Y] by doing [Z]"), side-by-side weak versus strong phrasing transformations, and engineering action verbs. The application is containerized with Gunicorn and deployed on Render with automated continuous integration from GitHub. The project demonstrates important concepts such as deterministic ATS modeling, modular frontend design, client-side vector reporting, and cloud deployment.

TABLE OF CONTENTS

1. Introduction	6
2. Problem Statement	6
3. Objectives	6
4. Scope of the Project	6
5. Technologies Used	7
6. System Requirements	7
7. System Architecture	7
8. System Design and Workflow	8
9. Database / Taxonomy Design	8
10. Implementation	9
11. Cloud Deployment	9
12. Monitoring and Analysis Pipeline	10
13. Testing	10
14. Results	11
15. Advantages	11
16. Limitations	11
17. Future Scope	11
18. Conclusion	12
19. References	12

1. INTRODUCTION

In modern talent acquisition workflows, automated Applicant Tracking Systems (ATS) process millions of job applications annually. Corporate recruitment pipelines filter out over 75% of candidate resumes before human recruiters ever inspect them. Traditional resume checkers often provide misleading evaluations by inflating scores based on superficial formatting rather than technical competence.

This project presents the Smart Resume Analyzer, an automated recruitment screening engine built with Python Flask, pypdf, and python-docx. The application accepts PDF and Word resumes, extracts plain text streams, verifies contact information, audits structural sections, and performs boundary-aware keyword matching against 6 synchronized technical roles.

The system operates completely deterministically without reliance on expensive, non-deterministic large language model APIs. Actionable feedback is provided using Google's X-Y-Z bullet formulation formula, accompanied by one-click client-side vector PDF audit report generation.

2. PROBLEM STATEMENT

Job seekers frequently face silent rejections from corporate hiring systems due to unparseable resume formats, omitted role keywords, or non-standard section headings. Existing online resume checkers suffer from critical flaws: they inflate scores by awarding high marks for basic contact details even when technical skills are absent, and their regex engines fail on programming languages with symbols (such as C++, C#, and Node.js).

The project addresses these challenges by creating a deterministic, recruiter-aligned ATS engine. Lookaround regular expressions eliminate token boundary traps, mathematical guardrails eliminate score inflation, and automated phrasing transformations guide applicants in crafting high-impact achievement statements.

3. OBJECTIVES

- Develop a functional web-based ATS resume screening application.
- Implement dual-format document parsing for PDF and DOCX files.
- Verify contact details (RFC email, telephone, LinkedIn, GitHub).
- Audit 5 standard resume sections (Contact, Summary, Education, Experience, Skills).
- Implement lookaround boundary regex matching solving token bleeding for C++, Node.js, and CI/CD.
- Synchronize technical keyword taxonomies across 6 engineering roles.
- Enforce relevance guardrails capping zero-match resumes at $\leq 25/100$ to eliminate score inflation.
- Provide actionable suggestions using Google's X-Y-Z bullet point formula.
- Implement client-side vector PDF report export using html2pdf.js.
- Deploy the application to Render cloud hosting with Gunicorn WSGI and automated CI/CD.

4. SCOPE OF THE PROJECT

The project covers the development, algorithmic design, and cloud deployment of an ATS resume screening platform. Its scope encompasses document extraction, section auditing, boundary tokenization, multi-factor weighted scoring, phrasing optimization, vector PDF generation, and continuous cloud deployment.

The architecture is designed for extensibility and can be extended in the future with OCR for scanned documents, semantic sentence embeddings, and custom job description parsing.

5. TECHNOLOGIES USED

Technology	Purpose	Role in Project
Python	Programming language	Backend engine and algorithmic scoring logic
Flask	Web framework	HTTP routes, upload handling, session controllers
pypdf	PDF parsing library	Plain text stream extraction from PDF resumes
python-docx	DOCX parsing library	XML paragraph and table text extraction from Word files
Gunicorn	WSGI application server	Production HTTP server with dynamic port binding
GitHub	Version control	Source code management and Render CI/CD deployment
Render	Cloud hosting	24/7 web application deployment with automatic SSL
HTML5 / CSS3	Frontend technologies	1440px widescreen dashboard and 10 modular views
html2pdf.js	Client-side PDF generator	One-click export of vectorized audit reports

6. SYSTEM REQUIREMENTS

6.1 Software Requirements

- Python 3.11 or compatible Python environment (Python 3.14 verified)
- Flask ≥ 3.0.0, Werkzeug ≥ 3.0.0, pypdf ≥ 5.0.0, python-docx ≥ 1.1.0
- Gunicorn ≥ 23.0.0 for production WSGI serving
- Git and GitHub account for source control and CI/CD
- Render account for cloud web service hosting
- Modern web browser (Chrome, Edge, Firefox, Safari)

6.2 Hardware Requirements

- A computer capable of running Python 3.11+ and Flask
- Minimum 2 GB RAM recommended for local document processing
- Stable internet connection for cloud deployment and CDN assets
- Sufficient storage for repository files and ephemeral document buffers

7. SYSTEM ARCHITECTURE

The application follows a decoupled multi-tier architecture. Candidates interact with the 1440px widescreen dashboard via their web browser. Ingestion routes validate uploaded documents, dispatching PDF streams to pypdf and DOCX streams to python-docx. Extracted text streams flow in parallel to section auditing and lookaround keyword scoring modules before reaching the Google X-Y-Z feedback generator.

Logical architecture:

- User Browser → Cloud Web Application (Render CDN)

- Cloud Web Application → Gunicorn WSGI Server → Flask Application Controller
- Flask Application → Document Extraction Engine (pypdf & python-docx)
- Extracted Text Stream → Section Auditor (Contact, Headers, Word Count)
- Extracted Text Stream → ATS Scorer (Lookaround Regex & Role Taxonomies)
- Audit & Keyword Metrics → Feedback Generator (Google X-Y-Z Formula)
- Response Payload → 10 Modular Frontend Components → Vector PDF Export

A visual architecture diagram has been compiled into `Architecture_Diagram.pdf` illustrating the complete data pipeline.

8. SYSTEM DESIGN AND WORKFLOW

8.1 Document Upload & Role Selection

1. The user opens the home page (`index.html`).
2. The user selects a target role from the 6 synchronized engineering profiles.
3. The user selects or drags-and-drops a resume file (PDF or DOCX).
4. The application verifies file extension and MIME type.
5. If the target role or file is missing, an on-page red notice banner is displayed.

8.2 Document Text Extraction

1. The upload endpoint (`/upload`) saves the file with `secure_filename`.
2. PDF files are parsed across all page stream buffers using `pypdf`.
3. DOCX files are parsed across paragraphs and table XML blocks using `python-docx`.
4. The raw text stream undergoes whitespace normalization and UTF-8 sanitization.
5. If text extraction returns empty (scanned image), an explicit notice prompts the user.

8.3 Section & Contact Verification

1. RFC-compliant regular expressions validate email syntax.
2. Standard phone patterns detect international and local phone numbers.
3. Social and portfolio URLs (LinkedIn and GitHub) are recognized.
4. The engine audits 5 core sections: Contact, Summary, Education, Experience, Skills.
5. Total word count, character count, and line density are calculated.

8.4 Lookaround Keyword Matching & Guarded Scoring

1. Lookaround regex patterns isolate keywords without substring bleeding.
2. The candidate text is matched against target role required skills.
3. The master skills taxonomy (50+ skills) is scanned for broader technical depth.
4. Action verbs and numerical impact metrics are inventoried.
5. Multi-factor scoring is computed and relevance guardrails are enforced.

9. DATABASE / TAXONOMY DESIGN

The project uses structured JSON storage (`data/job_roles.json`) for role-specific skill taxonomies. Six primary engineering profiles are synchronized:

Target Role	Required Core Keywords	Category	Description
Data Analyst	SQL, Python, Excel, Tableau, Power BI, Statistics, Pandas, ETL	Analytics	BI & ETL workflows
Web Developer	JavaScript, HTML, CSS, React, Node.js, Git	Frontend/Full	Web architecture

Target Role	Required Core Keywords	Category	Description
Cloud Engineer	AWS, Docker, Kubernetes, Linux, CI/CD, Terraform, Azure, GCP	Cloud & DevOps	Cloud infra & orchestration
Python Developer	Python, Django, Flask, SQL, Git, REST API, Docker, PostgreSQL	Backend	APIs & backend systems
Data Scientist	Python, Machine Learning, Scikit-learn, SQL, Statistics, Data Viz, Pandas	Data Science	Modeling & analytics

The master skills taxonomy contains 50+ normalized technical keywords, guaranteeing synchronized scanning across all software disciplines.

10. IMPLEMENTATION

10.1 Backend & Modular Pipeline Orchestration

The Flask backend handles document ingestion, security validation, and module coordination. The core/ package isolates extraction (parser.py), auditing (section_analyzer.py), scoring (ats_scorer.py), and recommendations (feedback_generator.py), returning a consolidated JSON dictionary to the results dashboard.

10.2 Lookaround Regex Boundary Engine

Standard regex `\b` boundaries fail on programming languages with symbols. The system implements custom lookaround patterns: `rf"(?!<![a-zA-Z0-9]){re.escape(keyword)}(?![a-zA-Z0-9])"`. This prevents 'Java' from matching inside 'JavaScript', 'C' from matching 'CSS', while accurately capturing 'C++', 'Node.js', and 'CI/CD'.

10.3 Relevance Guardrails & Scoring Logic

The multi-factor scoring formula weighs: Role Match (40%), Technical Depth (25%), Section Architecture (25%), and Contact Details (10%). Strict relevance guardrails are enforced: 0% match is capped at $\leq 25/100$; $< 25\%$ match is capped at $\leq 40/100$; $< 50\%$ match is capped at $\leq 55/100$. This eliminates artificial score inflation for unqualified resumes.

10.4 Actionable Feedback & Google X-Y-Z Formulation

The feedback generator dynamically creates tailored bullet points for missing skills, trains candidates on Google's X-Y-Z formula ("Accomplished [X], as measured by [Y], by doing [Z]"), and presents side-by-side weak versus strong phrasing transformations.

10.5 Componentized Presentation & Client-Side PDF Export

The frontend is divided into 10 reusable component templates in `templates/components/`. Report export uses `html2pdf.js` and `static/css/print.css`, generating print-ready vector PDF audit reports directly in the user's browser with zero server overhead.

11. CLOUD DEPLOYMENT

The application is prepared for cloud deployment using Gunicorn and GitHub. The source code is maintained in GitHub repository `subbaraju-cmd/Smart-Resume-Analyzer`. Render cloud builds and serves the application via `render.yaml` Blueprint and Procfile.

The production server uses Gunicorn with the Flask application and binds to the required host and port (\$PORT). Render provides automated continuous deployment from the GitHub main branch.

11.1 Deployment Flow

1. Develop and test the Flask application locally on port 5000.
2. Pin dependencies in requirements.txt (Flask $\geq$ 3.0.0, gunicorn $\geq$ 23.0.0).
3. Configure Procfile (web: gunicorn app:app) and render.yaml Blueprint.
4. Commit and push source code to GitHub repository.
5. Connect GitHub repository to Render cloud platform.
6. Build application and install dependencies using pip install -r requirements.txt.
7. Start production WSGI server using Gunicorn.
8. Verify live production endpoint (<https://smart-resume-analyzer-utj7.onrender.com>).

12. MONITORING AND ANALYSIS PIPELINE

An analytical dashboard provides a comprehensive view of candidate ATS metrics. Key evaluation parameters:

Evaluation Metric	Analytical Purpose & Weight
Role Match % (40%)	Calculates target-role keyword density using lookahead regex tokenization.
Technical Depth (25%)	Quantifies broader technical competencies against 50+ master skills.
Section Architecture (25%)	Audits structural headers: Contact, Summary, Education, Experience, Skills.
Contact Details (10%)	Validates RFC-compliant email, phone number, LinkedIn, and GitHub links.
Relevance Guardrails	Enforces mathematical caps preventing unqualified resumes from passing.

13. TESTING

The application was tested through local unit execution and live cloud deployment on Render. Functional areas were thoroughly checked:

Test Case	Expected Result	Observed Result	Status
PDF text extraction	Extracts clean text stream	Stream extracted successfully	Pass
DOCX text extraction	Extracts paragraph & table text	All XML blocks parsed	Pass
Unqualified resume (0% match)	Score capped at $\leq 25/100$	Score dropped from 65 to 24	Pass
Qualified resume (AI Eng)	High score with skill matches	Score 91/100, verified skills	Pass

Test Case	Expected Result	Observed Result	Status
C++ and Node.js token check	Exact match without symbol bleed	Lookaround pattern isolated token	Pass
Scanned image PDF upload	Displays on-screen red warning	Custom alert banner shown	Pass
RFC email verification	Validates standard syntax	Invalid addresses flagged	Pass
Vector PDF report export	Generates client-side PDF	Vector PDF generated cleanly	Pass
Gunicorn cloud startup	Binds to \$PORT on Render	HTTP 200 OK verified	Pass

14. RESULTS

The completed system provides a functional, cloud-ready ATS resume analyzer. The application supports dual-format document parsing, RFC contact auditing, boundary-aware token matching, anti-inflation scoring, Google X-Y-Z feedback generation, and client-side vector PDF reporting.

Empirical testing demonstrated that the anti-inflation relevance guardrails successfully eliminated false-positive passes: a non-technical resume evaluated against Web Developer dropped from an inflated 65/100 to 24/100. The production deployment on Render operates smoothly with sub-second response times.

15. ADVANTAGES

- Deterministic and reproducible evaluations with zero AI hallucination risk.
- Dual-format document parsing for both PDF and DOCX resume files.
- Lookaround regex engine prevents false substring matches on C++, Node.js, and CI/CD.
- Relevance guardrails mathematically eliminate artificial score inflation.
- Actionable phrasing improvements powered by Google's X-Y-Z formula.
- Client-side vector PDF export eliminates server rendering latency.
- 1440px widescreen dashboard designed with 10 modular, decoupled components.
- 24/7 cloud hosting on Render with automated GitHub CI/CD integration.
- Ephemeral document processing guarantees candidate privacy.

16. LIMITATIONS

- Scanned image-based resumes without embedded text require OCR preprocessing.
- Keyword matching is exact and does not automatically map semantic synonyms.
- Complex multi-column non-standard graphical tables may alter plain text reading order.
- Taxonomies are currently predefined across 6 major technical engineering roles.
- Does not yet support direct applicant tracking database storage for recruiters.

17. FUTURE SCOPE

- Integrate Tesseract OCR to support scanned image PDFs and multi-column visual resumes.
- Incorporate lightweight on-device sentence embeddings (e.g., MiniLM) for semantic synonyms.
- Add real-time job description pasting to allow dynamic custom keyword extraction.
- Implement cover letter cross-auditing against candidate resumes.

- Provide user authentication and persistent candidate audit history using PostgreSQL.
- Develop recruiter dashboard for batch resume screening and rank ordering.

18. CONCLUSION

The “Smart Resume Analyzer with AI-Based Feedback” project demonstrates how a web application can be developed using Python Flask and transformed into a production-ready recruitment screening engine through multi-format text parsing, lookaround boundary tokenization, relevance guardrails, actionable feedback generation, and cloud deployment with Gunicorn on Render.

The project provided practical understanding of the complete software engineering lifecycle, from algorithmic token matching and UI modularization to WSGI serving and operational deployment. The deterministic architecture provides a reliable, transparent foundation that can serve as a benchmark for automated resume evaluation without the unpredictability or cost of third-party language models.

19. REFERENCES

- Python Documentation — <https://docs.python.org/>
- Flask Documentation — <https://flask.palletsprojects.com/>
- pypdf Documentation — <https://pypdf.readthedocs.io/>
- python-docx Documentation — <https://python-docx.readthedocs.io/>
- Gunicorn Documentation — <https://docs.gunicorn.org/>
- GitHub Documentation — <https://docs.github.com/>
- Render Documentation — <https://render.com/docs>
- Google Resume Advice (X-Y-Z Formula) — <https://www.inc.com/bill-murphy-jr/google-recruiters-say-these-5-words-are-secret-to-a-perfect-resume.html>
- SkillOrbit project instructions and capstone requirements.

THANKYOU!